statnnet: Regression-Style Statistical Summaries for Neural Networks Fitted with nnet

by Andrew McInerney and Kevin Burke

Abstract The nnet package provides a stable and widely used implementation of single-hidden-layer feedforward neural networks in R, but its output is primarily concerned with fitting and prediction. The statnnet package extends fitted nnet objects with regression-style statistical summaries for small, parsimonious networks. It provides Wald-type summaries for individual weights and groups of input weights, partial covariate-effect plots with uncertainty estimates, and diagnostic information concerning optimisation and covariance estimation. The scope is deliberately restricted to supported models fitted using nnet; the package is not intended to provide general-purpose inference for deep or otherwise highly parameterised neural networks. This paper describes the statistical quantities computed by statnnet, explains the package design and its limitations, and demonstrates the workflow using an applied regression example.

1 Introduction

Feedforward neural networks (FNNs) are flexible non-linear models that are widely used for prediction and function approximation (LeCun et al., 2015). However, from a statistical perspective, they can also be viewed as nonlinear regression models, in which a vector of inputs is mapped to a conditional mean of a response through a sequence of weighted linear combinations and non-linear activation functions. In R, the **nnet** package (Venables and Ripley, 2002) provides a long-established implementation for fitting single-hidden-layer FNNs, with optional skip-layer connections and weight decay. An object returned by `nnet()` contains the fitted weights, fitted values, residuals, objective value, and convergence code, and can also contain a Hessian matrix. However, the standard output is primarily concerned with fitting and prediction and does not provide covariate-level standard errors, Wald-type summaries, analysis-of-variance-style tables, or covariate-effect summaries.

This creates a practical obstacle for statisticians who wish to use shallow FNNs as statistical modelling tools rather than only as predictive algorithms. In a classical regression workflow, a fitted model can be inspected using familiar functions such as `summary()`, `anova()`, `predict()`, and `plot()`. These functions help answer two questions that naturally arise when modelling the relationship between a response and a set of covariates: whether a covariate appears to contribute to the fitted model, and how the fitted response changes with that covariate. For a fitted FNN, answering these questions typically requires the user to extract weights manually, reconstruct the model matrix, compute approximate covariance matrices, write custom code for grouped summaries, or use model-agnostic interpretability tools. Model-agnostic approaches, such as Shapley values (Strumbelj and Kononenko, 2010) and locally interpretable model explanations (LIME) (Ribeiro et al., 2016), are useful for explaining fitted predictions across a wide range of model classes. However, they treat the fitted network as a black box and do not exploit the particular parameterisation of a shallow neural network as a non-linear regression model. The aim of **statnnet** is to add these model-based, regression-style summaries to a precisely defined subset of models fitted using **nnet**. Restricting the package to one fitting engine avoids treating materially different neural-network implementations as interchangeable and allows the parameter ordering, activation functions, objective, convergence information, and Hessian calculations used by **nnet** to be handled explicitly.

The statistical methodology underlying **statnnet** was developed in McInerney and Burke (2023), which treats suitably parsimonious shallow FNNs as statistical models, derives Wald-type tests for individual weights and covariate-level groups of weights, and introduces partial covariate-effect (PCE) plots as neural-network analogues of regression coefficients. The aim of the present paper is to describe the corresponding R implementation and to show how these summaries can be obtained through a workflow that will be familiar to users of standard R modelling functions. The **statnnet** package is a statistical-summary extension to **nnet**: it does not introduce a separate neural-network optimiser or attempt to provide a common representation for several fitting frameworks. Instead, it takes a supported fitted `"nnet"` object and constructs a `"statnnet"` object containing the additional information required for covariance estimation, grouped Wald-type summaries, PCE plots, and diagnostic checks. The resulting object can be inspected using S3 methods for printing, summaries, analysis-of-variance-style tables, plotting, and prediction.

By augmenting the established `"nnet"` workflow with a consistent set of S3 methods, **statnnet**

makes supported shallow FNNs behave more like standard statistical model objects in R. The paper is organised as follows. Section 2.2 introduces the FNN parameterisation and the statistical quantities used by **statnnet**. Section 2.3 positions the package relative to existing neural-network fitting and model-interpretability packages. Section 2.4 demonstrates the package workflow using an applied regression example. Section 2.5 summarises the supported model objects and S3 methods. Section 2.6 discusses practical limitations and appropriate interpretation of the resulting summaries. Section 2.7 concludes.

2 Statistical background

This section summarises the statistical quantities computed by **statnnet**. A fuller development of the methodology, including the derivation of the Wald-type summaries, simulation evidence, and practical recommendations, is given in McInerney and Burke (2023). Here, we introduce only the notation and definitions required to understand the package output.

The package is designed for supported single-hidden-layer models fitted using **nnet** that can be viewed as non-linear regression models. Let y_i denote the response for observation $i = 1, \ldots, n$, and let $x_i = (x_{i0}, x_{i1}, \ldots, x_{ip})^T$ denote the corresponding vector of covariates, where $x_{i0} \equiv 1$ is an intercept term. For a single-hidden-layer FNN with q hidden nodes, the conditional mean can be written as

$$\mathbb{E}(y_i \mid x_i) = \mathrm{NN}(x_i, \theta),$$

where

$$\mathrm{NN}(x_i, \theta) = \phi_o \left[\gamma_0 + \sum_{k=1}^{q} \gamma_k \phi_h \left(\sum_{j=0}^{p} \omega_{jk} x_{ij} \right) \right].$$

Here, ω_{jk} is the weight connecting input j to hidden node k, γ_k is the weight connecting hidden node k to the output node, γ_0 is the output intercept, and θ denotes the full vector of model parameters. The functions $\phi_h(\cdot)$ and $\phi_o(\cdot)$ are the hidden-layer and output-layer activation functions, respectively. For continuous responses, $\phi_o(\cdot)$ is typically the identity function; for binary responses, it is typically the logistic function.

The statistical calculations in **statnnet** use the fitting criteria implemented by **nnet** directly. Let $\delta \geq 0$ denote the scalar `decay` argument and let K be the number of fitted weights, including the hidden- and output-layer intercepts. For the supported linear-output regression model, `nnet()` minimises

$$Q_{\delta,G}(\theta) = \sum_{i=1}^{n} \{y_i - \mathrm{NN}(x_i, \theta)\}^2 + \delta \theta^T \theta.$$

For a binary factor response, `nnet()` uses a single logistic output with `entropy = TRUE` and minimises

$$Q_{\delta,B}(\theta) = -\sum_{i=1}^{n} [y_i \log\{\mathrm{NN}(x_i, \theta)\} + (1 - y_i) \log\{1 - \mathrm{NN}(x_i, \theta)\}] + \delta \theta^T \theta.$$

Thus, for the scalar `decay` argument considered here, **nnet** applies weight decay to all K fitted weights rather than excluding intercept weights. The component `value` is the relevant data-fitting criterion plus this weight-decay term.

Write $H_\delta = \nabla^2 Q_\delta(\hat{\theta})$ for the Hessian returned when `Hess = TRUE`. On the native **nnet** scale, the unpenalised curvature is

$$H_0 = H_\delta - 2\delta I_K,$$

where I_K is the $K \times K$ identity matrix. For binary entropy fitting, the corresponding likelihood-scale matrices are $J_0 = H_0$ and $J_\delta = H_\delta$. For Gaussian regression, the residual variance is estimated as $\hat{\sigma}^2 = n^{-1} \sum_{i=1}^{n} \{y_i - \mathrm{NN}(x_i, \hat{\theta})\}^2$, and the residual-sum-of-squares criterion is converted to the negative-log-likelihood scale using

$$J_0 = \frac{H_0}{2\hat{\sigma}^2}, \qquad J_\delta = \frac{H_\delta}{2\hat{\sigma}^2}.$$

The covariance matrix for the penalised estimate is then approximated by the sandwich expression

$$\widehat{\mathrm{Cov}}(\hat{\theta}) = J_\delta^{-1} J_0 J_\delta^{-1},$$

as described in McInerney and Burke (2023). When `decay = 0`, this reduces to the inverse observed-information approximation. The package checks the numerical behaviour of J_δ before computing the inverse.

The fitted parameter vector and covariance matrix are used to construct the Wald-type summaries and PCE plots described below. These quantities are conditional on the fitted architecture, value of `decay`, and local solution. They are not post-selection summaries and do not account for uncertainty arising from choosing the architecture, penalty, or initialisation.

Wald-type summaries

One aim of **statnnet** is to provide regression-style summaries for supported `"nnet"` fits. Given an estimate $\hat{\theta}$ and an estimated covariance matrix $\widehat{\mathrm{Cov}}(\hat{\theta})$, the package computes Wald-type summaries for both individual weights and groups of weights.

For an individual parameter θ_m, the null hypothesis

$$H_0 : \theta_m = 0$$

is summarised using the statistic

$$W_m = \frac{\hat{\theta}_m^2}{\widehat{\mathrm{Cov}}(\hat{\theta})_{mm}},$$

which is compared to a χ_1^2 reference distribution. This produces a parameter-level summary similar in form to the output from classical regression models. However, individual neural-network weights are usually difficult to interpret in isolation, because the effect of an input variable is distributed across multiple connections and depends on the rest of the fitted network.

For this reason, **statnnet** also provides covariate-level Wald-type summaries. For a single-hidden-layer FNN, the contribution of covariate x_j to the hidden layer is represented by the vector

$$\omega_j = (\omega_{j1}, \ldots, \omega_{jq})^T,$$

which contains all weights connecting covariate j to the hidden nodes. The package therefore computes a multiple-parameter Wald-type statistic for

$$H_0 : \omega_j = 0_q.$$

If S_j is the matrix selecting the elements of θ corresponding to ω_j, then the relevant covariance matrix is

$$\widehat{\Sigma}_{\omega_j} = S_j \widehat{\mathrm{Cov}}(\hat{\theta}) S_j^T.$$

The corresponding statistic is

$$W_j = \hat{\omega}_j^T \widehat{\Sigma}_{\omega_j}^{-1} \hat{\omega}_j.$$

Define the shrinkage matrix $A = J_\delta^{-1} J_0$. The statistic is compared to a chi-squared reference distribution with effective degrees of freedom

$$\tilde{q}_j = \mathrm{tr}(S_j A S_j^T),$$

which is approximately q when `decay` is small. It provides an input-level summary that is closer in spirit to testing whether a covariate contributes to the fitted model.

For categorical covariates represented by several dummy variables, **statnnet** extends this idea by testing all associated input nodes jointly. For example, if a categorical covariate with $l + 1$ levels is represented using l dummy variables, and the network has q hidden nodes, the corresponding grouped test involves lq input-to-hidden weights. Using the corresponding $lq \times K$ selection matrix in the expression above gives effective degrees of freedom that are approximately lq when `decay` is small. This gives an analysis-of-variance-style summary for categorical predictors, allowing the user to assess the contribution of the original covariate rather than each dummy variable separately.

Partial covariate effects

In addition to assessing whether a covariate contributes to the fitted model, it is useful to describe the nature of that contribution. For this purpose, **statnnet** provides PCE plots.

Partial dependence plots describe how the average fitted response changes as one covariate is varied while the remaining covariates are held at their observed values. For covariate j, the partial dependence function is estimated as

$$\overline{\mathrm{NN}}(x^{(j)}) = \frac{1}{n} \sum_{i=1}^{n} \mathrm{NN}(x_i^{(j)}, \hat{\theta}),$$

where $x_i^{(j)}$ denotes the covariate vector for observation i with the jth covariate set to the value $x^{(j)}$.

The PCE is a finite-difference version of this quantity. For a d-unit increase in covariate j, the PCE is defined as

$$\hat{\beta}(x^{(j)}, d) = \overline{\mathrm{NN}}(x^{(j)} + d) - \overline{\mathrm{NN}}(x^{(j)}).$$

This quantity has an interpretation close to a regression coefficient: it estimates the average change in the fitted response associated with a d-unit increase in a covariate. The value of d can be chosen by the user; common choices are $d = 1$ or d equal to one empirical standard deviation of the covariate.

For binary covariates, the PCE corresponds to the average change in the fitted response when the covariate changes from 0 to 1. For continuous covariates, plotting $\hat{\beta}(x^{(j)}, d)$ over a sequence of values shows how the covariate effect varies across the covariate range. When the PCE is approximately constant, the fitted relationship is approximately linear in that covariate; when it varies substantially, the fitted model suggests a non-linear effect.

Uncertainty intervals for the PCE can be computed using the approximate sampling distribution of $\hat{\theta}$. For the delta method, the gradient of the PCE with respect to $\hat{\theta}$ is used:

$$\nabla_{\hat{\theta}} \hat{\beta}(x^{(j)}, d) = \frac{1}{n} \sum_{i=1}^{n} \left[\nabla_{\hat{\theta}} \mathrm{NN}(x_i^{(j)} + d, \hat{\theta}) - \nabla_{\hat{\theta}} \mathrm{NN}(x_i^{(j)}, \hat{\theta}) \right].$$

Hence,

$$\widehat{\mathrm{Var}}\left(\hat{\beta}(x^{(j)}, d)\right) \approx \nabla_{\hat{\theta}} \hat{\beta}(x^{(j)}, d)^T \widehat{\mathrm{Cov}}(\hat{\theta}) \nabla_{\hat{\theta}} \hat{\beta}(x^{(j)}, d).$$

This gives an analytic approximation to the variance, and, therefore, the standard error of the PCE at each plotted covariate value. The corresponding normal-approximation intervals are pointwise intervals, not simultaneous confidence bands over the complete curve. Alternatively, pointwise percentile intervals can be obtained by simulating parameter vectors from the approximate sampling distribution of $\hat{\theta}$ and recomputing the PCE for each simulated set of parameters as in Mandel (2013). Both forms of interval are conditional on the selected architecture, value of `decay`, local solution, and approximate parameter covariance.

The package also provides a single point-estimate covariate effect by averaging the PCE over the observed values of the covariate:

$$\hat{\tau}_j = \frac{1}{n} \sum_{i=1}^{n} \hat{\beta}(x_{ij}, d).$$

This average partial-dependence contrast is available from `summary(object,effects = "partial")` and is accompanied by a standard error. Because its direct calculation averages over all combinations of the observed focal and remaining covariate values, its computational cost can be appreciable.

For continuous covariates, the default summary instead reports the faster average observed-row finite difference

$$\hat{\tau}_j^{\mathrm{row}} = \frac{1}{n} \sum_{i=1}^{n} \left\{ \mathrm{NN}(x_i + d e_j, \hat{\theta}) - \mathrm{NN}(x_i, \hat{\theta}) \right\},$$

where e_j selects covariate j. This estimand changes the focal covariate within each observed row and requires only $O(n)$ prediction evaluations. It generally differs from the average partial-dependence contrast when the fitted network contains interactions; the two coincide for additive fitted response functions. For binary and categorical covariates, the fixed reference contrasts have the same rowwise and partial-dependence interpretations. Together, the Wald-type summaries and PCE plots provide the two pieces of information usually sought from regression-style output: whether a covariate appears to contribute to the fitted model, and how the fitted response changes with that covariate.

3 Relationship to existing software

The role of **statnnet** is most clearly understood in relation to **nnet** itself. The `nnet()` function fits single-hidden-layer neural networks using the BFGS algorithm and provides methods for prediction, together with access to the fitted weights, residuals, objective value, convergence code, and Hessian. These quantities provide the computational basis for further statistical summaries, but **nnet** does not itself provide covariate-level Wald-type summaries or PCE plots with uncertainty estimates. The purpose of **statnnet** is to supply this additional layer for supported `"nnet"` objects.

This close dependence on **nnet** is a deliberate design choice. The package does not attempt to translate objects from several neural-network frameworks into a common representation, and it does not claim that the same covariance calculations are appropriate for arbitrary architectures or fitting objectives. Instead, the supported parameterisation, activation functions, weight ordering, objective, and convergence information are tied directly to those used by **nnet**. This narrower interface makes it possible to validate the required model information and to issue warnings when a fitted network lies

Table 1: Information available from a native **nnet** fit and after augmentation by **statnnet**.

Task	nnet	statnnet
Model fitting	Fits by BFGS.	Retains the fit; no refitting.
Default summary	Architecture and connection weights.	Effects and grouped Wald summaries.
Parameter covariance	Not generally supplied.	Penalised sandwich estimate.
Covariate-level assessment	Not provided.	Grouped Wald summaries.
Covariate-effect description	Not provided.	PCE summaries and plots.
Numerical diagnostics	Convergence; optional Hessian.	Adds covariance checks.
Prediction	Provided by 'predict.nnet()'.	Delegates to the retained fit.

outside the intended statistical setting.

As previously mentioned, there are also several R packages for model-agnostic interpretability, including packages that provide partial dependence plots, accumulated local effects, variable-importance measures, surrogate models, or Shapley-value-based explanations. Examples include packages such as **pdp** (Greenwell, 2017), **iml** (Molnar et al., 2018), **DALEX** (Biecek, 2018), and **vip** (Greenwell and Boehmke, 2020). These tools are valuable because they can be applied to many different fitted model classes. Their model-agnostic nature means that the same interpretability method can often be used for random forests, boosted trees, support vector machines, neural networks, and other predictive models. However, this generality also means that they are not designed to use the specific likelihood-based parameterisation of a shallow FNN. They are primarily designed to explain fitted prediction functions rather than to provide regression-style summaries based on the fitted neural-network parameters.

The approach in **statnnet** is different. It uses the fitted FNN parameterisation to compute Wald-type summaries, covariance-based standard errors, and PCE uncertainty intervals. The resulting output is therefore closer in spirit to the output from classical statistical models, such as `lm()` and `glm()`, than to purely model-agnostic explainability tools. This distinction is important. A model-agnostic partial dependence plot can describe how predictions change as a covariate varies, but it does not usually provide a covariate-level Wald-type summary based on the fitted neural-network parameters. Similarly, a variable-importance measure can rank predictors by predictive contribution, but it is not usually a model-based standard error for a covariate-effect estimate. **statnnet** provides these statistically motivated summaries for the specific case of supported single-hidden-layer models fitted using **nnet**.

Direct comparison with `nnet`

The contribution of **statnnet** is most visible by comparing the information available before and after a fitted `"nnet"` object is augmented. The native `summary()` method prints the network dimensions, fitting options, and fitted connection weights. It does not change the fitted model, but it also does not provide a covariance matrix, covariate-level tests, or effect summaries. After conversion, **statnnet** retains the original fit and adds the statistical quantities shown in Table 1.

4 Using statnnet

We now demonstrate the main workflow in **statnnet** using the insurance data considered in McInerney and Burke (2023). This section is intended to show the package as it would typically be used: a parsimonious, single-hidden-layer FNN is fitted using **nnet**, augmented as an object of class `"statnnet"`, and then inspected using familiar statistical modelling functions. The aim is not to repeat the full applied analysis from McInerney and Burke (2023), but to show how the package moves from a fitted neural network to regression-style summaries, covariate-level tests, PCE plots, network visualisations, and predictions.

The main function is `statnnet()`, which takes a fitted `"nnet"` object and constructs the additional quantities required for statistical summaries:

```
statnnet(object, formula, data, ...)
```

The resulting object retains the fitted `"nnet"` model and stores the model matrix, response, likelihood information, covariance estimates, Wald-type summaries, covariate-effect estimates, and diagnostic quantities required by the package methods. Model fitting remains the responsibility of **nnet**; **statnnet** supplies a statistical-summary layer for the supported fitted object.

The current implementation is deliberately restricted to parsimonious single-hidden-layer FNNs fitted to tabular data, with continuous or binary response variables. The model is treated as a likelihood-based non-linear regression model, with a Gaussian likelihood for continuous responses and a Bernoulli likelihood for binary responses. Input variables may be continuous, binary, or categorical, with categorical variables represented through the corresponding dummy variables in the model matrix. The package does not support deep architectures, multiple hidden layers, or fitted objects produced by other neural-network packages.

Data

The data contain information on $n = 1338$ insurance beneficiaries. The response variable, `charges`, is the total medical expenses charged to an insurance plan. The explanatory variables are the beneficiary's age, sex, body mass index, number of children covered by the plan, smoking status, and region of residence. The version of the data included in **statnnet** is already centred, scaled, and dummy encoded, so the factor variables initially appear as binary input columns.

```
library(statnnet)
library(nnet)

data("insurance", package = "statnnet")

names(insurance)

#> [1] "age"             "sexmale"         "bmi"             "children"
#> [5] "smokeryes"       "regionnorthwest" "regionsoutheast" "regionsouthwest"
#> [9] "charges"

head(insurance[, c("age", "sexmale", "bmi", "children", "smokeryes", "charges")])

#>          age sexmale        bmi    children smokeryes    charges
#> 1 -1.4382265       0 -0.4531506 -0.90827406         1  0.2984722
#> 2 -1.5094011       1  0.5094306 -0.07873775         0 -0.9533327
#> 3 -0.7976553       1  0.3831636  1.58033487         0 -0.7284023
#> 4 -0.4417824       1 -1.3050431 -0.90827406         0  0.7195739
#> 5 -0.5129570       1 -0.2924471 -0.90827406         0 -0.7765118
#> 6 -0.5841316       0 -0.8073542 -0.90827406         0 -0.7856145
```

To demonstrate the package's factor grouping, we reconstruct the original region factor from its mutually exclusive indicator columns and remove those columns before fitting. The resulting model matrix is unchanged apart from retaining the mapping of its three region columns to one formula term.

```
insurance_analysis <- insurance
insurance_analysis$region <- factor(
  ifelse(
    insurance$regionnorthwest == 1,
    "northwest",
    ifelse(
      insurance$regionsoutheast == 1,
      "southeast",
      ifelse(insurance$regionsouthwest == 1, "southwest", "northeast")
    )
  ),
  levels = c("northeast", "northwest", "southeast", "southwest")
)
insurance_analysis[c(
  "regionnorthwest", "regionsoutheast", "regionsouthwest"
)] <- NULL
```

For this example, we model `charges` using all available covariates:

```
insurance_formula <- charges ~ .
```

The variables include continuous covariates, such as `age` and `bmi`; binary indicator covariates, such as `sexmale` and `smokeryes`; and the reconstructed `region` factor. The example therefore illustrates both input-level summaries and joint term-level summaries when a categorical covariate expands to several model-matrix columns.

Fitting the neural network

We first fit a single-hidden-layer FNN using `nnet()`. Since the objective is non-convex, we use several random initialisations and retain the converged fit with the smallest fitted objective value:

```
set.seed(1237019)

nnet_fits <- replicate(
  10,
  nnet(
    insurance_formula,
    data = insurance_analysis,
    size = 2,
    linout = TRUE,
    decay = 0.01,
    maxit = 1000,
    Hess = TRUE,
    trace = FALSE
  ),
  simplify = FALSE
)

converged <- vapply(nnet_fits, function(fit) fit$convergence == 0, logical(1))
stopifnot(any(converged))

objective <- vapply(nnet_fits, function(fit) fit$value, numeric(1))
objective[!converged] <- Inf
nn <- nnet_fits[[which.min(objective)]]
```

The fitted model has two hidden nodes and uses the **nnet** weight-decay argument with `decay = 0.01`. The convergence check prevents a fit that reached the iteration limit from being selected solely because of its objective value. The statistical summaries below are conditional on the retained architecture, penalty, and local solution; they do not account for the choice among initialisations.

Native `nnet` summary

Before applying **statnnet**, the native summary displays the fitted architecture, fitting options, and connection weights:

```
summary(nn)

#> a 8-2-1 network with 21 weights
#> options were - linear output units  decay=0.01
#>  b->h1 i1->h1 i2->h1 i3->h1 i4->h1 i5->h1 i6->h1 i7->h1 i8->h1
#>  -0.67   0.65  -0.11   0.02   0.12   1.77  -0.08  -0.16  -0.22
#>  b->h2 i1->h2 i2->h2 i3->h2 i4->h2 i5->h2 i6->h2 i7->h2 i8->h2
#>  14.30  -0.06  -0.06  -3.50   0.14 -14.55  -0.19  -0.10  -0.14
#>   b->o  h1->o  h2->o
#>   0.92   2.32  -2.07
```

This output is useful for checking the model structure, but the individual connection weights do not directly describe the overall contribution or effect of a covariate. The following sections show the additional statistical information obtained from the same fitted model.

Creating a `"statnnet"` object

The object `nn` is a fitted `"nnet"` object. It can be augmented with the model information required by **statnnet**:

```
snn <- statnnet(
  object = nn,
  formula = insurance_formula,
  data = insurance_analysis,
  response = "continuous"
)
```

The fitted `"nnet"` object does not always retain all of the information required to reconstruct the statistical model, so the original formula and training data are supplied explicitly. The constructor checks the fitted architecture, response type, convergence code, dimensions of the reconstructed model matrix, and availability of the quantities required for covariance estimation.

The resulting object retains the original `"nnet"` fit and the model information required by the statistical methods, including the reconstructed model matrix, response, weight-decay value, covariance estimate, grouped Wald-type summaries, PCE summaries, and numerical diagnostics. These components are normally accessed through the methods below rather than directly.

Printing the fitted network

The `print()` method gives a compact overview of the fitted FNN:

```
print(snn)
```

```
#> statnnet: statistical summary of a fitted nnet model
#> Formula: charges ~ .
#> Architecture: 8-2-1; 21 weights; n = 1338
#> Response: continuous; decay: 0.01
#> nnet convergence code: 0
#> Covariance: available (reciprocal condition number 5.62e-06)
```

The printed output reports the model call, response type, number of observations, number of input nodes, number of hidden nodes, and the value of the ridge penalty. This gives a quick check that the fitted network has the intended architecture before the statistical summaries are examined.

Regression-style summary

The `summary()` method gives the main regression-style summary produced by **statnnet**:

```
summary(snn)
```

```
#> statnnet summary
#> Formula: charges ~ .
#> Response: continuous; architecture: 8-2-1
#>
#> Average predictive finite-difference effects
#>   variable                                contrast         value step
#>        age average observed-row finite difference -1.724186e-16    1
#>    sexmale                                   1 - 0  1.000000e+00    1
#>        bmi average observed-row finite difference -2.097737e-16    1
#>   children average observed-row finite difference -8.764235e-17    1
#>  smokeryes                                   1 - 0  1.000000e+00    1
#>     region                 northwest - northeast            NA   NA
#>     region                 southeast - northeast            NA   NA
#>     region                 southwest - northeast            NA   NA
#>     estimate  std_error    conf_low    conf_high
#>   0.31363077 0.01112587  0.29182447  0.335437066
#>  -0.04643133 0.02006795 -0.08576379 -0.007098873
#>   0.14386456 0.01080637  0.12268446  0.165044654
#>   0.05046739 0.00994552  0.03097453  0.069960252
#>   1.97032148 0.02579351  1.91976713  2.020875825
#>  -0.03099674 0.02894994 -0.08773758  0.025744088
#>  -0.07110343 0.02911544 -0.12816865 -0.014038217
#>  -0.09547673 0.02876126 -0.15184776 -0.039105687
```

```
#>
#> Grouped input-to-hidden Wald summaries
#>       term                                      model_columns n_weights
#>        age                                                age         2
#>    sexmale                                            sexmale         2
#>        bmi                                                bmi         2
#>   children                                           children         2
#>  smokeryes                                          smokeryes         2
#>     region regionnorthwest, regionsoutheast, regionsouthwest         6
#>  effective_df wald_chisq      p_value status
#>      1.997897  25.809166 2.478255e-06     ok
#>      1.997447   4.602134 9.993031e-02     ok
#>      1.989125 106.151550 8.683343e-24     ok
#>      1.999329  14.285650 7.898082e-04     ok
#>      1.841755 165.817646 6.588393e-37     ok
#>      5.985367   9.691684 1.373538e-01     ok
```

The first table reports average predictive finite-difference effects over the observed covariate rows, with pointwise standard errors and intervals when covariance estimation is available. For factors it reports one contrast for each non-reference level. The separate grouped Wald table tests all model-matrix columns belonging to each formula term jointly. Thus, the summary provides two pieces of information that are commonly sought in regression modelling: descriptive effect estimates and a test of whether a term appears to contribute to the fitted model.

For example, in the insurance data, variables such as `age`, `bmi`, `children`, and `smokeryes` are expected to contribute strongly to the fitted response. The summary output allows these effects to be inspected in a form that is closer to a classical regression summary than to the raw weight output usually obtained from neural-network software.

By default, `summary()` focuses on covariate-level summaries. The more computationally intensive average partial-dependence effects defined in Section 2.2 can be requested explicitly:

```
summary(snn, effects = "partial")
```

Single-parameter summaries for individual neural-network weights can also be displayed:

```
summary(snn, weights = TRUE)
```

These individual-weight summaries are mainly diagnostic. For interpretation, the covariate-level summaries and the PCE plots are usually more informative than individual input-to-hidden-layer weights.

Analysis-of-variance-style summary

The `anova()` method provides covariate-level Wald-type summaries:

```
anova(snn)
```

```
#> Grouped input-to-hidden Wald summaries
#>       term                                      model_columns n_weights
#>        age                                                age         2
#>    sexmale                                            sexmale         2
#>        bmi                                                bmi         2
#>   children                                           children         2
#>  smokeryes                                          smokeryes         2
#>     region regionnorthwest, regionsoutheast, regionsouthwest         6
#>  effective_df wald_chisq      p_value status
#>      1.997897  25.809166 2.478255e-06     ok
#>      1.997447   4.602134 9.993031e-02     ok
#>      1.989125 106.151550 8.683343e-24     ok
#>      1.999329  14.285650 7.898082e-04     ok
#>      1.841755 165.817646 6.588393e-37     ok
#>      5.985367   9.691684 1.373538e-01     ok
```

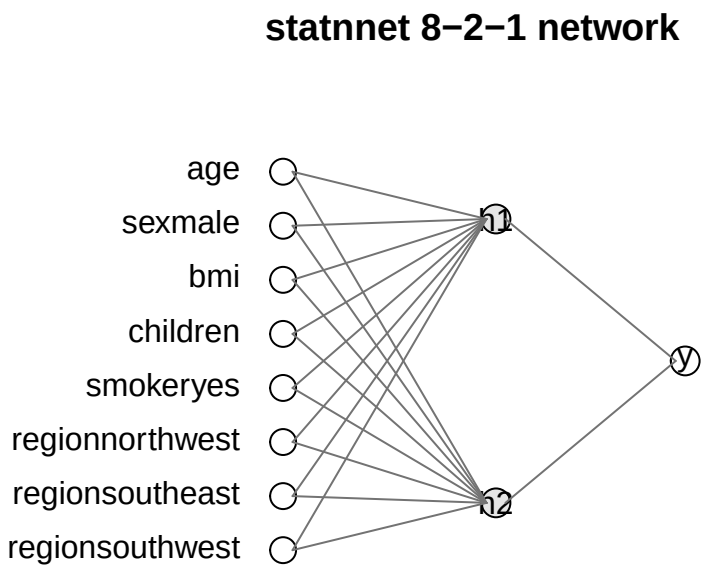


Figure 1: Fitted neural-network architecture for the insurance data. The diagram shows the input nodes, hidden nodes, output node, and fitted connections; annotations are suppressed here to keep the dense diagram legible.

For continuous and binary covariates, this corresponds to testing the group of weights connecting the covariate to the hidden layer. For categorical covariates represented by multiple dummy variables, such as region, the method tests all associated input nodes jointly. This gives a summary that is analogous in spirit to an analysis-of-variance table, where the contribution of the original covariate is assessed rather than the contribution of each dummy variable separately.

This is particularly useful when interpreting factor variables. For example, rather than separately inspecting the dummy variables corresponding to different regions, the `anova()` method gives a single covariate-level summary for region as a whole.

Network visualisation

The fitted architecture can be visualised using `plotnn()`:

```
plotnn(snn, annotate = "none")
```

The plot displays the input nodes, hidden nodes, output node, and fitted connections. By default, input-to-hidden edges are annotated with their fitted weight estimates. Individual-weight Wald p-values can be displayed instead using `annotate = "p_value"`; edges whose p-values are below `alpha` are highlighted. These edge-level annotations are diagnostic, while `anova()` provides the intended term-level summaries.

The p-value annotation and significance threshold can be selected using:

```
plotnn(snn, annotate = "p_value", alpha = 0.01)
```

Partial covariate-effect plots

The `plot()` method produces PCE plots. For a continuous covariate such as `bmi`, the PCE plot shows how the estimated effect of a d-unit increase in BMI varies across the observed BMI range. The default

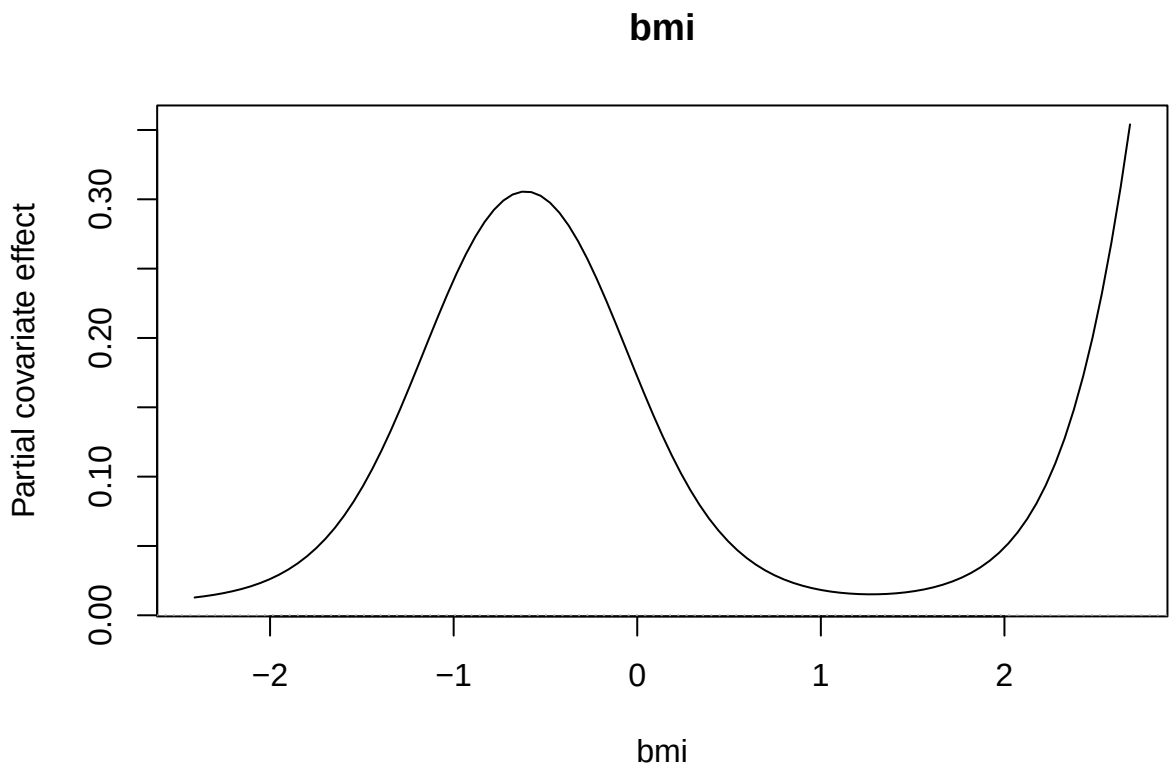


Figure 2: Partial covariate-effect plot for body mass index in the insurance data. The plot shows the estimated change in fitted charges associated with a change in standardised body mass index over the observed covariate range.

grid is restricted so that both the baseline value and its d-unit comparison remain within the observed univariate range.

```
plot(snn, variable = "bmi", uncertainty = "none")
```

If the PCE is approximately constant, the fitted model suggests an approximately linear effect for that covariate. If the PCE varies across the covariate range, the fitted model suggests a non-linear effect. This is one of the main advantages of the PCE plot: it gives an effect-size interpretation while still allowing the fitted neural network to represent non-linear relationships.

For binary covariates, the PCE corresponds to the average change in fitted response when the covariate changes from 0 to 1. For example, the effect of smoking status can be visualised using

```
plot(snn, variable = "smokeryes", uncertainty = "none")
```

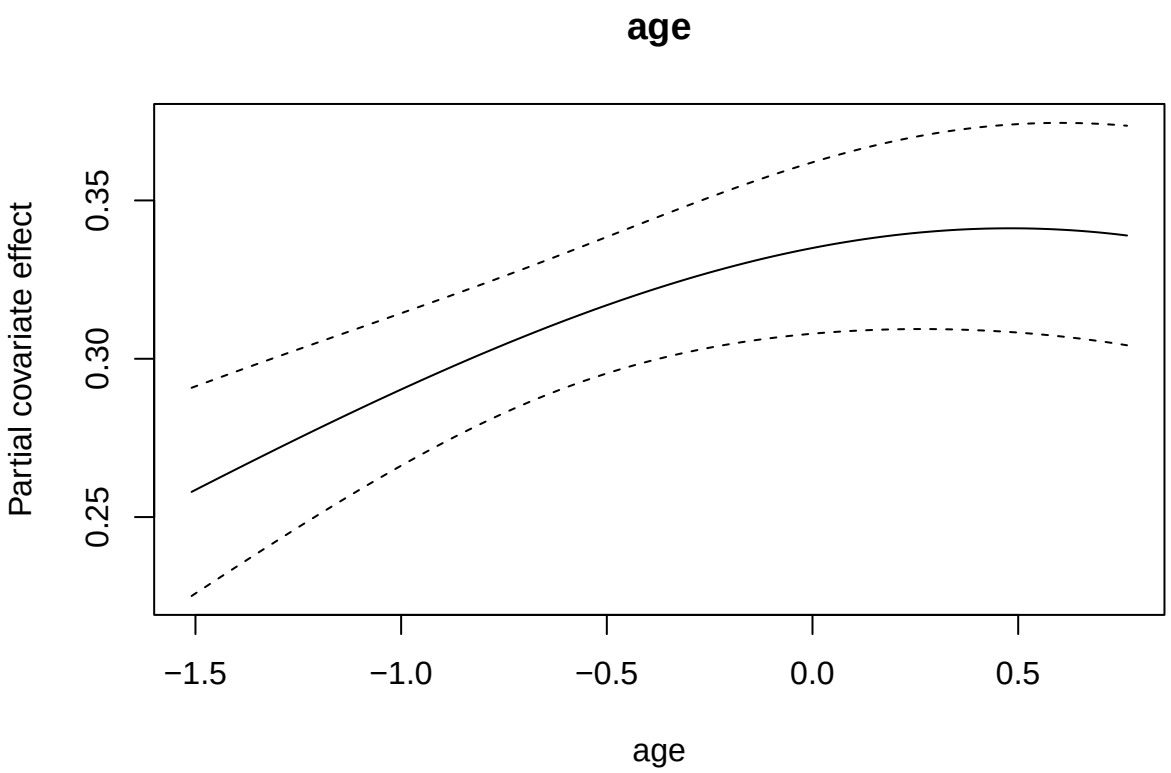
Several covariates can be plotted at once:

```
plot(
  snn,
  variable = c("age", "bmi", "children", "smokeryes"),
  uncertainty = "none"
)
```

The uncertainty intervals are computed using the approximate sampling distribution of the fitted parameters. The uncertainty calculation is selected using the `uncertainty` argument. For example, the delta-method intervals are obtained using:

```
plot(
  snn,
```

```
  variable = "age",
  uncertainty = "delta"
)
```



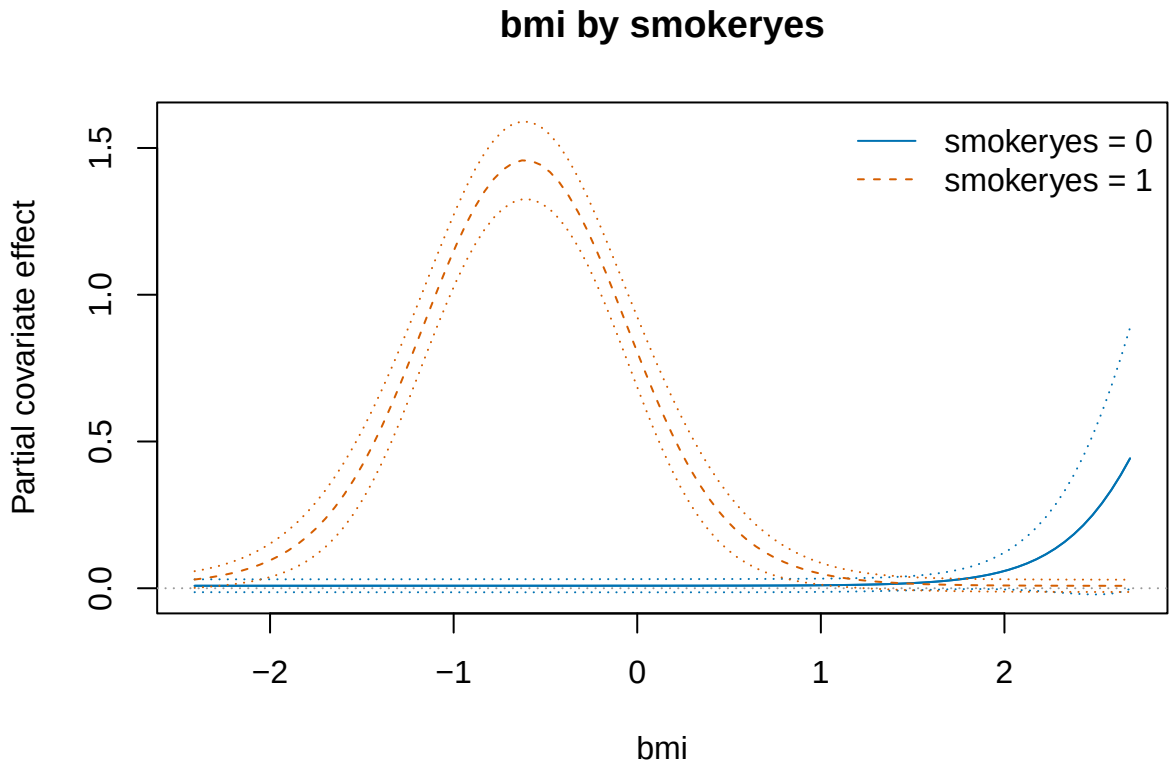
Alternatively, simulation from the approximate sampling distribution of the fitted parameters can be used:

```
plot(
  snn,
  variable = "age",
  uncertainty = "simulation"
)
```

Visualising interactions

Because FNNs can represent interactions between covariates, it can also be useful to examine whether a PCE changes across the values of another covariate. For example, the effect of `bmi` can be plotted separately for different smoking-status groups when the fitted object and plotting method support grouped PCE displays:

```
plot(
  snn,
  variable = "bmi",
  by = "smokeryes",
  uncertainty = "delta"
)
```



If the resulting PCE curves differ substantially between the two smoking-status groups, this suggests that the fitted network has captured an interaction between `bmi` and `smokeryes`. This type of plot is exploratory, but it is useful for communicating how the fitted neural network changes across different regions of the covariate space.

Prediction

The `predict()` method returns fitted values for the training data when no new data are supplied:

```
fitted_values <- predict(snn)

head(fitted_values)

#>         [,1]
#> 1  0.4063545
#> 2 -0.8490492
#> 3 -0.6325964
#> 4 -0.6407389
#> 5 -0.6515446
#> 6 -0.6627258
```

Predictions for new observations can be obtained using the `newdata` argument:

```
new_insurance <- insurance_analysis[
  1:5,
  setdiff(names(insurance_analysis), "charges")
]

predict(snn, newdata = new_insurance)

#>         [,1]
#> 1  0.4063545
```

Table 2: Main user-facing functions and methods in statnnet.

Function	Purpose
‘statnnet()‘	Validates and augments an ‘nnet‘ fit.
‘print()‘	Prints architecture and diagnostics.
‘summary()‘	Reports effects and grouped Wald summaries.
‘anova()‘	Reports grouped Wald summaries.
‘vcov()‘	Returns the covariance estimate.
‘plot()‘	Plots PCEs and pointwise intervals.
‘plotnn()‘	Draws an annotated network.
‘predict()‘	Delegates prediction to the retained fit.

```
#> 2 -0.8490492
#> 3 -0.6325964
#> 4 -0.6407389
#> 5 -0.6515446
```

This preserves a prediction interface for `"statnnet"` objects that is consistent with the underlying `"nnet"` fit. The same object can therefore be used both for interpretation and prediction.

5 Supported objects and methods

This section records the supported-model contract and user-facing methods; the corresponding workflow was demonstrated in Section 2.4.

Supported model contract

statnnet supports single-hidden-layer `"nnet"` models fitted to tabular data. Continuous responses require a linear output and are treated as Gaussian non-linear regression models; binary responses require a single logistic output with entropy fitting and are treated as Bernoulli regression models. Skip-layer connections, softmax or censored outputs, multiple output nodes, and objects fitted by other neural-network packages are outside this scope.

Inputs may be continuous, binary, or categorical. When the original formula and data are supplied, the mapping from the model-matrix columns to the original covariates is retained so that all dummy variables associated with a factor can be summarised jointly. The original `"nnet"` fit is also retained, allowing prediction to be delegated to `predict.nnet()`.

Requirements and diagnostic checks

The input to `statnnet()` must inherit from class `"nnet"`, and the original training data must always be supplied. The formula may be omitted only when it can be recovered from an `"nnet.formula"` object. The constructor checks the architecture, response type, convergence code, and agreement between the reconstructed model matrix and the fitted network.

Before computing covariance-based summaries, the package checks the **nnet** convergence code and the availability and numerical behaviour of the Hessian. If the penalised curvature is effectively singular or cannot be inverted reliably, the affected covariance-based summaries are not reported. These checks do not establish every regularity condition, but they prevent clearly unsuitable fits from being summarised without notice.

User-facing functions and methods

The main user-facing functions and methods are summarised in Table 2. Together, these functions add statistical summaries to a fitted `"nnet"` object while preserving the established fitting and prediction workflow.

nnet therefore remains responsible for fitting and prediction, while `statnnet()` and the `"statnnet"` methods provide the additional statistical summaries and visualisations.

6 Practical guidance and limitations

The covariance-based summaries are intended for small, parsimonious networks whose fitted parameters are locally well identified. Increasing the hidden-layer size can improve flexibility, but it also increases the number of weights and the likelihood of redundant hidden nodes or near-singular curvature. The `decay` value can improve numerical stability and is incorporated into the covariance calculation exactly as described in Section 2.2; both the hidden-layer size and `decay` should therefore be reported and examined in sensitivity analyses. Convergence of `nnet()` alone is insufficient: the covariance diagnostics should also be checked before interpreting standard errors, Wald-type statistics, or uncertainty intervals.

The fitting objective is non-convex, so different initial values can lead to different local solutions. Using several initialisations reduces the risk of reporting a poor solution, but does not account for the uncertainty introduced by selecting among them. Similarly, the reported reference distributions are conditional on the chosen architecture and `decay` value. If these are selected using the same data, the output should not be interpreted as post-selection inference; the selection procedure and the stability of the resulting summaries should be reported separately.

Individual neural-network weights should be interpreted cautiously. A single input-to-hidden-layer weight is only one component of the fitted function, and its meaning depends on the remaining weights in the network. In addition, hidden nodes can be permuted without changing the fitted input-output function, and sign or label symmetries can lead to equivalent parameterisations. For this reason, **statnnet** reports individual-weight summaries mainly as diagnostic quantities. In most applications, input-level summaries, covariate-level summaries, and PCE plots will be more useful for interpretation.

Categorical covariates should generally be interpreted at the covariate level rather than only at the dummy-variable level. When a factor is represented by several dummy variables, separate input-level summaries can be difficult to interpret in isolation. The `anova()` method therefore groups the associated input nodes and provides a covariate-level Wald-type summary. This gives an output closer in spirit to an analysis-of-variance table for a standard regression model.

PCE plots should be interpreted as summaries of the fitted model, not as causal effects. A PCE describes how the fitted response changes when a covariate is varied while the remaining covariates are averaged over their observed values. As with partial dependence plots more generally, interpretation can be more difficult when covariates are strongly correlated or when the plot evaluates combinations of covariate values that are rare in the observed data. For default continuous PCE curves, **statnnet** keeps both finite-difference endpoints within the observed univariate covariate range. User-supplied evaluation values and average effects are not restricted in this way and can still involve extrapolation, particularly near a boundary or with a large finite-difference step. The plots are therefore best viewed as descriptive tools for understanding the fitted prediction function.

The inferential summaries produced by **statnnet** do not replace model checking or predictive validation. Users should still assess predictive performance using appropriate validation procedures, examine the sensitivity of the fitted model to architecture and penalty choices, and consider whether the chosen neural network is appropriate for the scientific or applied question. The package makes fitted shallow FNNs more interpretable, but the quality of the resulting interpretation depends on the quality and stability of the fitted model.

7 Conclusion

The **statnnet** package provides a statistical-summary layer for supported single-hidden-layer neural networks fitted using **nnet**. Rather than introducing a new optimiser or a common interface for several neural-network frameworks, the package augments a fitted `"nnet"` object with the information required for regression-style summaries, diagnostic checks, and covariate-effect visualisations. In this way, **statnnet** makes a deliberately restricted class of shallow neural networks behave more like standard statistical model objects in R.

The package implements regression-style summaries, Wald-type summaries for individual weights and groups of weights, analysis-of-variance-style summaries for covariates, PCE plots with uncertainty estimates, architecture diagrams annotated with statistical summaries, and prediction methods for new observations. These tools complement **nnet** by providing outputs that are more familiar to users of classical regression models.

The statistical methodology underlying these summaries is developed in McInerney and Burke (2023). The contribution of the present article is to describe the corresponding R implementation and demonstrate how the methodology can be used in practice through a coherent package workflow. By combining the flexibility of a parsimonious single-hidden-layer network with regression-style output,

statnnet provides a practical interface for statisticians who wish to use supported `"nnet"` models as statistical modelling tools rather than purely predictive algorithms.

Future development will remain centred on the **nnet** setting, with possible extensions including alternative uncertainty estimators and additional diagnostics for assessing the stability of fitted models. Support for deep-learning frameworks or arbitrary neural-network architectures is not an objective of the package. More generally, **statnnet** provides a focused extension for integrating small `"nnet"` models more naturally into the statistical modelling workflow in R.

8 Acknowledgements

This publication has emanated from research conducted with the financial support of Science Foundation Ireland under Grant number 18/CRT/6049. The second author was supported by the Confirm Smart Manufacturing Centre (https://confirm.ie/) funded by Science Foundation Ireland (Grant Number: 16/RC/3918). For the purpose of Open Access, the authors have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

Bibliography

P. Biecek. Dalex: Explainers for complex predictive models in r. *Journal of Machine Learning Research*, 19 (84):1–5, 2018. URL https://jmlr.org/papers/v19/18-416.html. [p5]

B. M. Greenwell. pdp: An R package for constructing partial dependence plots. *The R Journal*, 9(1): 421–436, 2017. doi: 10.32614/RJ-2017-016. URL https://doi.org/10.32614/RJ-2017-016. [p5]

B. M. Greenwell and B. C. Boehmke. Variable importance plots—an introduction to the vip package. *The R Journal*, 12(1):343–366, 2020. doi: 10.32614/RJ-2020-013. [p5]

Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539. [p1]

M. Mandel. Simulation-based confidence intervals for functions with complicated derivatives. *The American Statistician*, 67(2):76–81, 2013. doi: 10.1080/00031305.2013.783880. [p4]

A. McInerney and K. Burke. Feedforward neural networks as statistical models: Improving interpretability through uncertainty quantification. 2023. [p1, 2, 5, 15]

C. Molnar, B. Bischl, and G. Casalicchio. iml: An r package for interpretable machine learning. *JOSS*, 3(26):786, 2018. doi: 10.21105/joss.00786. URL https://joss.theoj.org/papers/10.21105/joss.00786. [p5]

M. T. Ribeiro, S. Singh, and C. Guestrin. Why should i trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 1135–1144, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342322. doi: 10.1145/2939672.2939778. URL https://doi.org/10.1145/2939672.2939778. [p1]

E. Strumbelj and I. Kononenko. An efficient explanation of individual classifications using game theory. *The Journal of Machine Learning Research*, 11:1–18, 2010. [p1]

W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. Springer, New York, fourth edition, 2002. ISBN 0-387-95457-0. [p1]

Andrew McInerney
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
ORCiD: 0000-0002-2348-2159
andrew.mcinerney@ul.ie

Kevin Burke
University of Limerick
Department of Mathematics and Statistics
Limerick, Ireland
kevin.burke@ul.ie